Search Typeahead System

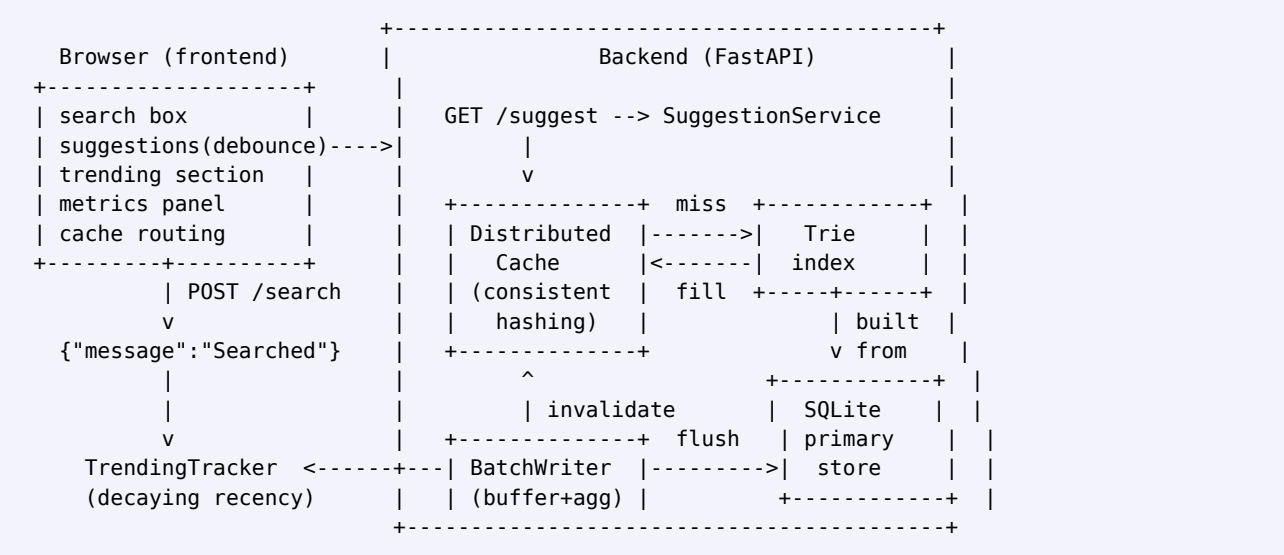
Project Report

A backend-focused typeahead system: a Trie suggestion index, a consistent-hashed distributed cache, recency-aware trending, and batched writes.

Author	Varun Mundada
Roll No.	24BCS10326
Course	SST-2028 · HLD101 — Build a Search Typeahead System
Repository	github.com/milesmoralis2411/hld_assignment
Date	June 2026

1. Architecture

The system is a search-as-you-type backend with a thin web UI. Suggestions are served from an in-memory **Trie** index fronted by a **distributed cache** (several logical LRU+TTL nodes addressed by a **consistent-hash ring**). Search submissions feed a **recency-aware trending** tracker immediately and are persisted in **batches** to a durable **SQLite** primary store, which is also the source the Trie is built from. This keeps reads fast and shields the database from per-search write pressure.

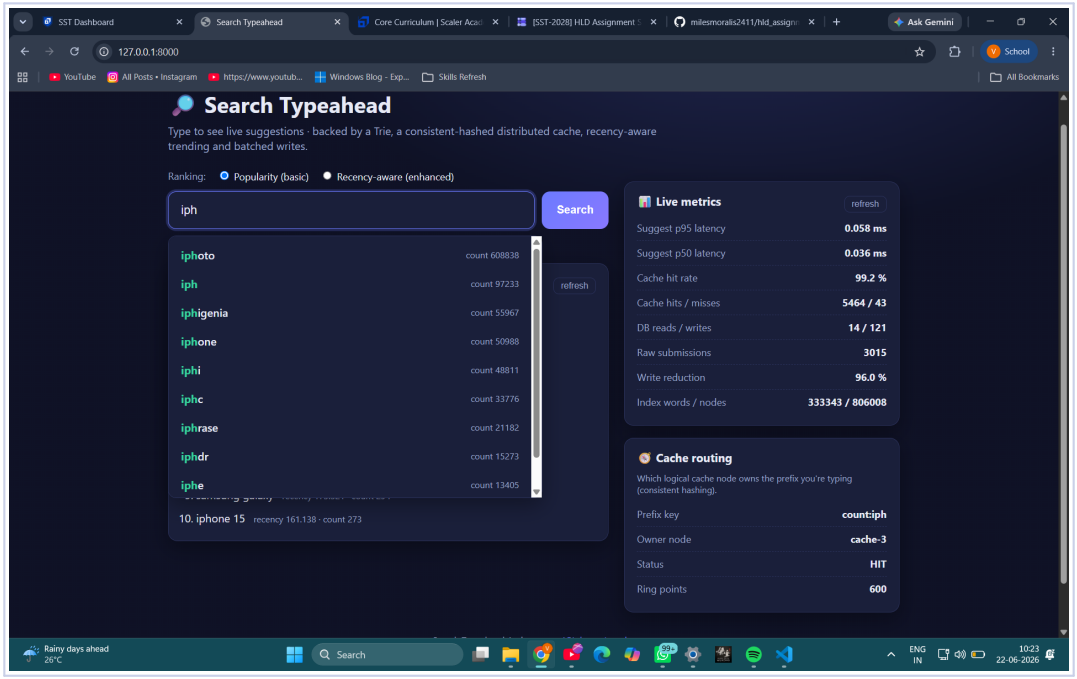


Data-flow paths

- **Read** (GET /suggest): cache → on miss, the Trie returns a candidate pool → rank (by count, or by count+recency) → store the result back in the cache.
- **Write** (POST /search): record recency **immediately** (so trending reacts instantly) and **append the count update to a buffer**; return {"message": "Searched"}.
- **Flush** (background task): drain the buffer, **aggregate repeated queries**, persist in **one transaction**, refresh the Trie, and invalidate affected cache keys.

Backend components (app/)

Module	Responsibility
consistent_hash.py	Hash ring with virtual nodes; routes a prefix key to a cache node.
cache.py	One logical cache node: LRU + per-entry TTL + hit/miss stats.
cache_cluster.py	N cache nodes behind the ring = the distributed cache.
trie.py	Prefix index; precomputes top-N pools for shallow prefixes (bounded memory).
store.py	SQLite primary store; counts every DB read and write.
trending.py	Exponentially time-decayed recency scores.
batch_writer.py	Buffer + aggregate + periodic / size-based flush.
metrics.py	Rolling latency window producing p50/p95/p99.
service.py	Wires it together (read / write / flush / trending / metrics).
main.py	FastAPI routes + lifespan bootstrap + static frontend.



The running UI: typeahead suggestions, live metrics and consistent-hash cache routing.

2. Dataset source and loading instructions

Source (real, open)

The **Google Web Trillion Word Corpus** unigram counts — `count_1w.txt`, published by Peter Norvig (https://norvig.com/ngrams/). It contains **333,333 real keywords with real corpus frequencies** and is free to use. "Keywords" is one of the entry types the assignment explicitly allows, and the counts are real frequencies (no aggregation needed).

- **Size:** all 333,333 rows by default (≫ the 100k minimum). Cap with `TYPEAHEAD_DATASET_LIMIT` or `--limit` (file is sorted by descending frequency).
- **Format:** raw file is `word<TAB>count`; the loader converts it to the assignment's `query,count` CSV at `data/queries.csv`.

Loading instructions

The dataset is fetched and loaded **automatically on first run** (`python run.py`) — it just needs internet access the first time. To do it explicitly:

```
python -m scripts.download_dataset            # -> data/queries.csv (all rows)
python -m scripts.download_dataset --limit 150000   # top-150k by frequency
```

On startup the CSV is bulk-loaded into SQLite (once), then the in-memory Trie is built from the store (333,333 words / ~806,000 nodes, ready in ~10-15 s on first run, ~3-4 s thereafter).

Using a different dataset / offline

- Drop any `query,count` CSV at `data/queries.csv` (e.g. an AOL query log or a Kaggle e-commerce search-terms set) and delete `data/typeahead.db` so it reloads. If the file has no counts, aggregate duplicates into counts first.

• If the download cannot reach the network on first run, the app falls back to a reproducible synthetic generator so it still runs (a message is printed).

3. API documentation

Base URL `http://127.0.0.1:8000`. Interactive, auto-generated docs are also served at `/docs` (Swagger UI).

Endpoint	Purpose	Notes
GET /suggest?q=<prefix>&ranking=count|recent	Fetch up to 10 suggestions whose query starts with the prefix.	ranking=count (default) = by overall count; ranking=recent = count + decaying recency.
POST /search {"query": "..."}	Dummy search submission.	Records recency now, buffers the count increment; returns {"message": "Searched"}.
GET /trending	Currently trending queries.	Recency-aware (time-decayed).
GET /cache/debug?prefix=<p>	Show consistent-hash routing for a key.	Returns owning node, key hash, ring info and HIT/MISS.
GET /metrics	Latency / cache / DB / batch metrics.	p50/p95/p99, hit rate, write reduction.
POST /admin/flush	Force a batch flush now.	Handy in demos so writes show immediately.
GET /health	Liveness check.	{"status": "ok"}.

Example responses

`GET /suggest?q=iph`

```
{
  "prefix": "iph", "ranking": "count", "source": "store",
  "suggestions": [
    {"query": "iphoto", "count": 608838},
    {"query": "iph", "count": 97233},
    {"query": "iphigenia", "count": 55967},
    {"query": "iphone", "count": 50988}
  ]
}
```

`GET /cache/debug?prefix=iph`

```
{
  "key": "count:iph", "owner_node": "cache-3", "cache_status": "HIT",
  "total_nodes": 4, "virtual_nodes_per_node": 150, "total_points_on_ring": 600
}
```

`GET /trending`

```
{"trending": [{"query": "python", "recency_score": 12.5, "count": 17610578}]}
```

4. Design choices and trade-offs

In-memory Trie with bounded precomputation

Suggestions need the top-K completions of a prefix. A Trie gives O(prefix length) traversal. The catch is broad prefixes (“a”, “ip”) that fan out to huge subtrees, so we **precompute a top-N candidate pool only for shallow nodes** (depth ≤ 4); those expensive lookups become O(1), while deep prefixes (tiny subtrees) are computed on demand. This bounds memory instead of storing a list on all ~806k nodes. **Trade-off:** a recency-surging query with a very low count may not be in a broad prefix's pool until its count rises — acceptable, because searching it also raises its count and it still appears in the trending section.

Distributed cache via consistent hashing

Each prefix-ranking key (e.g. `count:iph`) is routed by a consistent-hash ring to one of N logical cache nodes (LRU + TTL). It is modelled in-process so the whole system is one-command-runnable, but the behaviour is real: even key spread via 150 virtual nodes per node, per-shard stats, and only ~1/N keys remapped when a node is added/removed. **Trade-off:** TTL means suggestions can be briefly stale after a write; we additionally invalidate the changed query's short prefixes on flush to tighten this.

Recency without permanent bias

Trending uses an **exponentially decayed counter** per query (`score = score × 0.5^(Δt / half_life) + 1`). It reacts instantly to bursts but the boost fades on its own, so a short-lived spike does not stay over-ranked. It is O(1) per update with no sliding-window buffers to sweep. The enhanced ranking blends it as `final = log1p(count) + recency_weight × recency` — log-compressing popularity so recency stays meaningful whether counts are in the tens or the billions (this dataset's counts reach ~2.3×10^10). **Trade-off:** `half_life` and `recency_weight` tune freshness vs. stability.

Batched writes

`POST /search` never writes to the DB synchronously — it buffers. A background flusher drains on size or interval and **aggregates repeated queries** (50× “iphone” → one +50 row-write), collapsing many writes into few. **Failure trade-off:** the buffer is in-memory, so a crash before a flush loses un-flushed submissions; mitigations (kept off to favour low write latency) would be a durable append-only log / persistent queue (Kafka, fsync'd WAL). On graceful shutdown the remaining buffer is flushed.

Why SQLite + FastAPI

Zero-setup (“easy to run locally”) yet a real transactional store and a real async HTTP framework with auto-generated API docs. The access pattern (bulk load, single-row upserts in a transaction on flush) maps cleanly onto any RDBMS, and the cache layer onto Redis, if scaled out. All tunables live in `app/config.py` (overridable via environment variables).

5. Performance report

Methodology

Driven over HTTP by `scripts/benchmark.py`: first 3,000 searches (to exercise batched writes), then 5,000 suggestion reads over a Zipf-biased prefix mix at concurrency 16. Default config: 4 cache nodes × 150 virtual nodes, TTL 30 s, batch size 200, flush interval 2 s, dataset = 333,333 real keywords (806k Trie nodes). Numbers below are a sample local run (Windows 11, Python 3.12) and vary by machine.

Suggestion read latency

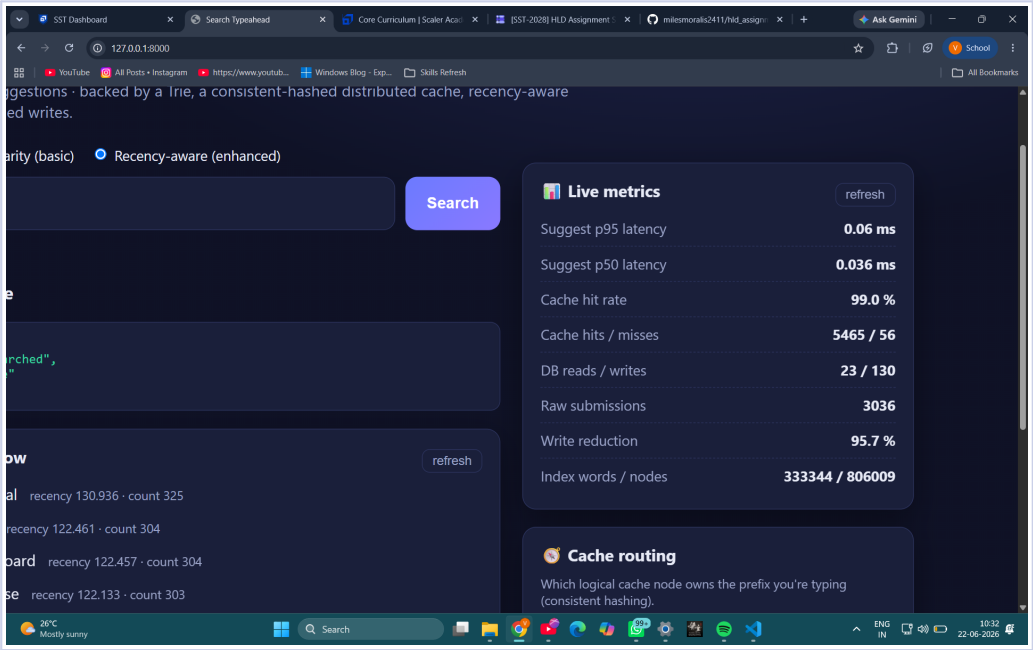
Metric	Client-measured (incl. HTTP)	Server-side (engine)
p50	32.90 ms	0.036 ms
p95	37.43 ms	0.057 ms
p99	41.67 ms	0.111 ms
avg	33.02 ms	0.036 ms
max	136.43 ms	0.545 ms

Server-side latency excludes HTTP/loopback/threading overhead, so it is the truest measure of the engine: **p95 ≈ 0.06 ms** over a 333k-keyword index. Client figures are dominated by the Python loopback round-trip under 16 threads, not the engine.

Distributed cache, database and write reduction

Area	Result
Cache hit rate	99.29% (5,464 hits / 39 misses across 4 nodes)
Cache key spread	cache-0: 1981 · cache-1: 1483 · cache-2: 1088 · cache-3: 912
DB reads / writes	14 reads / 121 writes (despite 5,000 reads + 3,000 searches)
Raw search submissions	3,015
DB row-writes performed	121 (writes saved: 2,894)
Write reduction (batching)	**95.99%** over 13 flushes

The cache + in-memory Trie absorb almost all read traffic (only 14 DB reads), and batching collapses ~3,000 submissions into 121 row-writes — a **96% reduction** in write pressure.



Live metrics panel from the running app (real dataset).

Interpreting the trade-offs

- **Latency vs. freshness (cache TTL):** higher TTL → higher hit rate / lower latency but staler post-write reads (mitigated by prefix invalidation on flush).
- **Throughput vs. durability (batching):** bigger batches / longer intervals → fewer DB writes but a larger in-memory window lost on a crash.
- **Freshness vs. stability (recency):** shorter half-life / higher weight → trendier but jumpier ranking; decay prevents permanent over-ranking.
- **Memory vs. cold-prefix latency (Trie precompute depth):** deeper precomputation → faster broad-prefix lookups but more memory.